

Mini-Project 2

Kristina Kjellgren, Miranda Lindgren & Lina Müller

2024-10-17

Introduction

Handwritten digit classification plays a crucial role in various applications, from postal sorting systems to automated data entry. As technology advances, the ability to accurately and efficiently recognize handwritten characters has become increasingly important. With the rise of machine learning and artificial intelligence, effective digit classification methods are essential for improving automated systems and reducing human error.

This report examines the challenges of classifying grayscale images of handwritten digits (0 to 9), which are typically represented as pixel matrices. Given the high dimensionality of image data, traditional methods can be computationally intensive and inefficient. Therefore, approaches like low rank approximation that use linear algebra and dimensionality reduction techniques can enhance classification performance while minimizing computational resources.

By exploring these methods, we aim to demonstrate their functionality in recognizing and classifying handwritten digits.

Approach

This project focuses on the classification of handwritten digits using Singular Value Decomposition (SVD), a standard technique in pattern recognition. The goal is to develop an algorithm that can classify a set of test digits, based on a training dataset, by decomposing each digit's image matrix into singular vectors and singular values.

The first step in this task involves analyzing the training data, which consists of a set of comprised grayscale images of handwritten digits (0 to 9). Each image is represented as a 28×28 -pixel matrix, which is then flattened into a vector of length 784. These vectors are subsequently stacked to form a larger matrix representing the entire dataset. Given that the training set consists of 240,000 images, the resulting matrix has dimensions $784 \times 240,000$, where each column corresponds to a vectorized image of a handwritten digit. The training data is used to form a matrix A_d for each digit d where the columns of A_d are the vectorized images of that digit. Using SVD, each matrix A_d is decomposed into three components: U , Σ and V^T . The matrix U and V contain the orthogonal basis vectors for the column and row spaces of A_d , respectively. In this project, the columns in U are specifically used to derive "singular images", which represent some of the most significant features of each digit.

The next step involves dimensionality reduction. As most columns of A_d are expected to be nearly linearly dependent (due to different handwriting styles of the same digit), the leading

singular vectors are sufficient to capture the core structure of each digit's subspace. Thus, a low-rank approximation of A_d is used to improve computational efficiency and reduce noise in the classification process. The Σ component is a diagonal matrix with singular values in descending order. Since the singular values rapidly decay for these matrices, retaining only the largest singular values and corresponding singular vectors is often sufficient to capture the essential structure of the data. This truncated decomposition enables us to approximate A_d using fewer dimensions while preserving most of the relevant information. This leads to faster computations and less storage.

To classify an unseen test digit, represented by a vector δ , one computes how well it can be approximated by the singular vectors of each digit class. This is done by solving a least-squares problem for each class, minimizing the residual between the test digit and its projection onto the corresponding space. Mathematically, this involves finding the coefficients that best express δ as a linear combination of the singular vectors of each class. To test how well the algorithm performs, a separate test set is used. This set contains digits with known labels, which allows for a comparison between the algorithm's predictions and the actual digits. The performance of the algorithm is evaluated based on how accurately it recognizes the digits in the test set. For each digit class d , the least-squares problem can be formulated as *Equation 1*):

$$\min_x ||U_k x - \delta||_2 \quad \text{Equation 1)}$$

U_k is the matrix of leading singular vectors for a digit d and x represents the coefficients of the linear combination. The residual derived from *Equation 1*), quantifies how well δ fits the subspace of the digit class. The digit class with the smallest residual is chosen as the predicted label for δ .

In this project, this SVD-based classification method is implemented, using the provided training and test datasets. Finally, performance can be evaluated by testing the algorithm on all provided test images and comparing the predicted label with their true labels.

Method

In this method, the residual is used for classification. This can be derived from the least square problem *Equation 1*). The minimum of x is obtained when:

$$U_k x = \delta \quad \text{Equation 2)}$$

Since the columns of U_k are orthonormal, the equality in *Equation 3*) holds true:

$$U_k^T U_k = I \quad \text{Equation 3)}$$

When multiplying *Equation 2*) with U_k^T the expression is transformed as below and an expression for x is found:

$$U_k^T U_k x = U_k^T \delta \Leftrightarrow x = U_k^T \delta \quad \text{Equation 4)}$$

When Equation 4) is substituted back into Equation 1), the residual becomes:

$$||U_k U_k^T \delta - \delta||_2 \quad \text{Equation 5)}$$

Equation 4) can also be written as:

$$||(U_k U_k^T - I)\delta||_2 \quad \text{Equation 6)}$$

In norms, changing sign of the expression will not change the length. As a result, the residual can be expressed as:

$$||-(I - U_k U_k^T)\delta||_2 = ||(I - U_k U_k^T)\delta||_2$$

The final expression to calculate the residual in this project is therefore as follows:

$$residual = ||(I - U_k U_k^T)\delta||_2 \quad \text{Equation 7)}$$

This can be written in python code as a function *residual* shown below:

```
def residual(Uk_submatrix, test_image_vector):
    identity_matrix = np.eye(784)
    projection = np.dot(Uk_submatrix, Uk_submatrix.T)
    difference = np.dot(identity_matrix - projection, test_image_vector)
    return np.linalg.norm(difference, axis=0)
```

The *extract_indices* method shown below is used to search for a specific digit in *DataLabel* and then return the index of the searched digit as well as how many times it occurs. The values returned by the function are further used in the method *set_up_testmatrix* to find the indices where the searched digit appears, allowing for the extraction of corresponding columns from a larger dataset and returning them as a matrix. Using this method, matrices for the digits 0-9 can be created, and then Singular Value Decomposition (SVD) can be performed to further train the program to recognize different handwritten digits.

```
def extract_indices(searchDigit, DataLabel, nSearch):
    indices = (DataLabel.flatten()[:nSearch] == searchDigit)
    indVec = np.argwhere(indices).flatten()
    numInd = len(indVec)
    return numInd, indVec
```

Furthermore, we want to perform SVD to determine how much data is necessary for the program to recognize the desired digits. By obtaining the singular values *S* and the singular vectors *U* for the different digits and plotting them in graphs and as images, one can determine when the significance of higher indices becomes less important for the overall impression of the image. From the matrix generated by *set_up_testmatrix* shown below, we can extract *S* and *U*, and then test how the algorithm for classifying handwritten digits works.

```
def set_up_testmatrix(searchDigit, DataSet, DataLabel, numData):
    nSearch = 100000
    numInd, indVec = extract_indices(searchDigit, DataLabel, nSearch)
    if numInd < numData
        raise ValueError("Did not find enough indices")
    A = DataSet[:, indVec[:numData]]
    A = np.float32(A)
    return A
```

To classify the test image the function *residual* is implemented. It computes the residual between the test image and the projection of the image onto a reduced subspace consisting of the first k singular vectors obtained from the SVD, this is represented by *Uk_submatrix* in the code. The projection is calculated by multiplying *Uk_submatrix* with its transpose. Then, the residual can be calculated by subtracting the projection from a identity matrix of size 784x784 and multiplying with the test image vector. The norm of the residual is returned and this gives a measure of which digit the image most likely represents by determining which digit has the subspace that produces the smallest residual.

Now for the classification algorithm we start off by looping through different values of k and then initialize the matrix *residual_matrix* to store all residuals for all digits and test images. Then we have another loop where we compute the residuals between the training images and the test images for all digits (0-9). This is done by extracting the first *num_columns* columns of the U matrix for that particular digit. So, for the first iteration it will extract the first five singular vectors (which are the columns in U) for digit 0. This creates a reduced subspace for that digit. Then the residual between the training image (*Uk_digit*) and the test image (*TestMat*) for that digit is computed and stored in *residuals_matrix[digit, :]* for the current digit.

So, each column in *residuals_matrix* corresponds to a single test image and the rows of that column consists of the residual between the test image and the columns of the matrix U that corresponds to a digit (0-9). The smallest residual for each test image is the predicted label (which is the predicted digit) for that test image. The index of the smallest residual for each test image is located with the NumPy function *argmin* and stored in the array *predicted_labels*. Since each row of *residuals_matrix* corresponds to a different digit (0-9), the index of the row will give us the predicted digit itself for that test image.

The overall success percentage for the current value of k is calculated by taking the mean value of how many times the *predicted_labels* correlates to the true labels (*TestLabel*), which is performed with a mask that will return True if they are the same. By multiplying with 100 we get the percentage and can store it in the array *overall_success_percentage*.

```

for num_columns in range(min_k, max_k + 1):
    residuals_matrix = np.zeros([10, num_test_images])
    for digit in range(10):
        Uk_digit = U_matrices[digit][:, :num_columns]
        residuals_matrix[digit, :] = residual(Uk_digit, TestMat)
    predicted_labels = np.argmin(residuals_matrix, axis=0)
    overall_success_percentage[num_columns - min_k] = np.mean(predicted_labels ==
TestLabel) * 100

```

By looping over all test images, we have generated predicted labels. Then the algorithm loops over the ten digits again, but this time to identify which test image belongs to digit d . The test images that are not labeled as the current digit d are filtered out. The success percentage for digit d is calculated by extracting the predicted labels for the test images that were labeled as digit d and comparing it to their true labels, to check if they were classified correctly. The result is multiplied by 100 and stored in the array *success_percentage_per_digit*.

```

for digit in range(10):
    digit_indices = (TestLabel == digit)[0]
    success_percentage_per_digit[digit, num_columns - min_k] =
np.mean(predicted_labels[digit_indices] == TestLabel[0, digit_indices]) * 100

```

In summary, we have created a classification algorithm for handwritten digits that first calculates the residuals between each test image against the subspace of each digit. Finds the digit that generates the smallest residual for that test image and classifies the test image with that predicted label. Finally, it checks if the test images were labelled accurately and calculates the overall success percentage for all test images, as well as the success percentage for each individual digit.

Results

By plotting the first 2000 images, which is represented by numdata=2000 in the Python code, we obtain graphs A and B in Figure 1. Graphs A and B show that the singular values decrease significantly at higher indices, which also is visualized in Figures 2A and 2B. Figures 2A and 2B only show values up to index 30 (k), providing a clearer view of that the significance becomes negligible at $k = 15$. To further observe how the significance of the singular values diminishes, one can study the normalized values in Figure 1C where a very steep curve is clearly visible for small indices.

As observed in Graph 1AB, the significance of the singular values decreases rapidly as the index increases. This can also be observed in Figure 3, where the first three singular images, U , for digits 3 and 8 are plotted. U_1 for both 3 and 8 shows the clearest image of the corresponding handwritten digit, while the images become blurrier for U_2 and U_3 .

The reconstruction of all 10 digits using only the first column of U_k , which corresponds to the first singular value, can be observed in Figure 4. All digits have a clear and distinct shape but lacks some definition around the edges and curves.

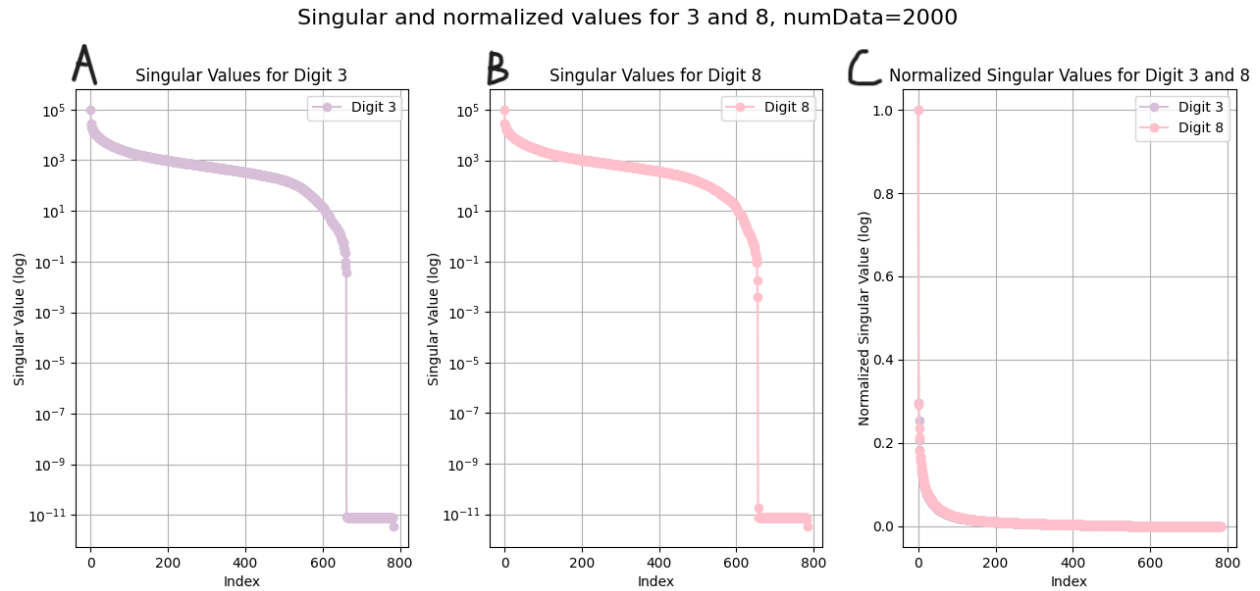


Figure 1 - Singular values for digit 3 and 8 plotted in different graphs and normalized singular values for the same digits combined in a third graph. All plots were made using 1000 training images. The Y-axis is on a logarithmic scale and represents the value of the singular values, while the X-axis represents the value of k .

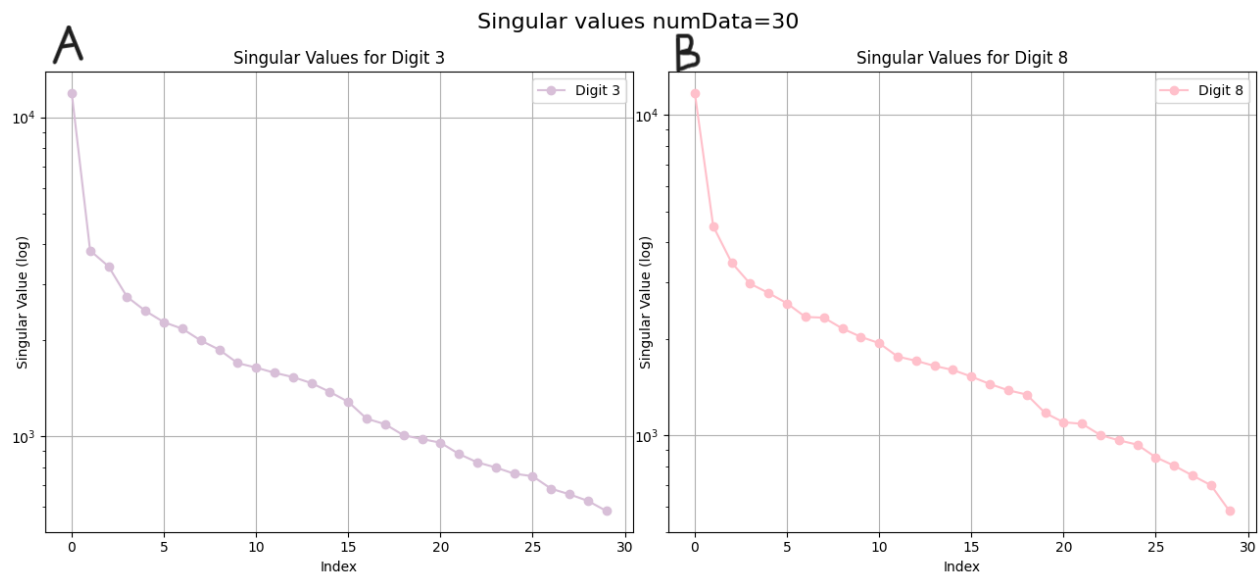


Figure 2 - Singular values for digit 3 (A) and 8 (B) made using 30 training images. The Y-axis is on a logarithmic scale and represents the value of the singular values, while the X-axis represents the value of k .

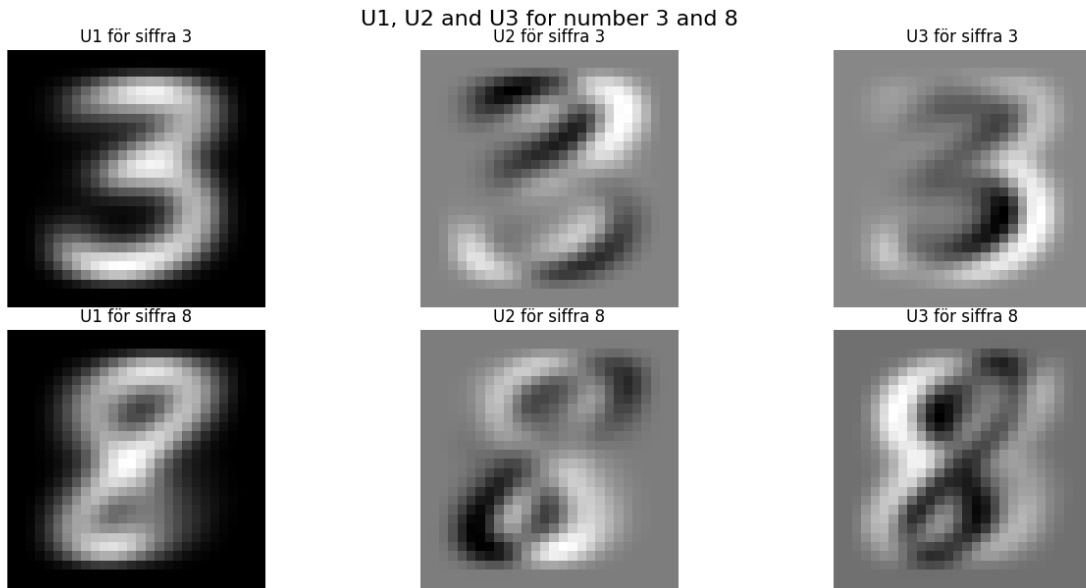


Figure 3 - Plot of the three first singular images $U1$, $U2$ and $U3$ for digit 3 and 8.

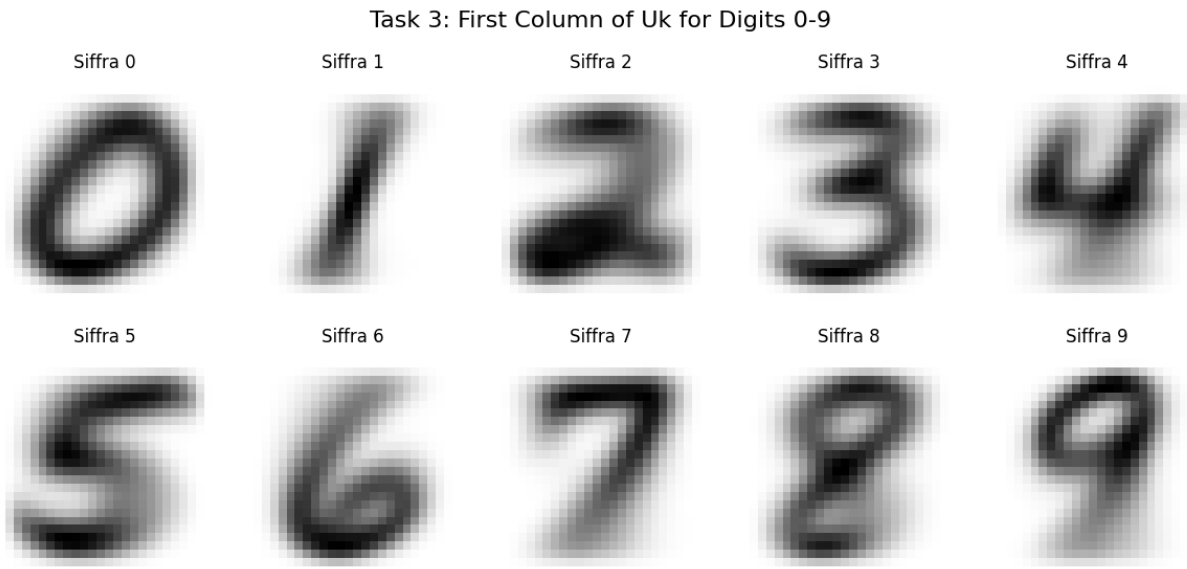


Figure 4 - Plot of the first column of U_k for all ten digits.

In Figure 6, the success percentage of the trained program for digits 0-9 is shown, where k ranges from 5 to 15. *Success percentage* refers to how well the SVD-based algorithm, trained on training images, performs in identifying the different numbers from the test images. The figure illustrates how the different graphs behave at various values of k , with all digits achieving a higher success percentage at $k = 15$ compared to $k = 5$. Digit 1 reaches the highest final success percentage, exceeding 98%, as also noted in Table 1, while digit 8 has the lowest, with a peak success percentage just above 90%. Both digits 8 and 2 show a significant increase from $k = 5$ to $k = 15$, whereas digits 0, 1, and 6 remain relatively stable. Digit 5 experiences a sharp drop between

$k = 10$ and $k = 12$, which sets it apart from all other trends in the graph. Figure 5 represents the overall success percentage for all digits combined, where a higher value of k corresponds to a higher success percentage. The exact value of the combined success percentage for each k can be observed in Table 2. In Table 1 as well as in Table 2, it is observed that there is an upward trend where the higher the value of k , the higher is also the success percentage. However, in Table 1, a deviation from the trend is observed where the success percentage is higher for $k=7$ than for $k=8$.

Table 1 - Percentage of success of the SVD-based algorithm for digit 1 using k (number of basis functions) between 5 and 15.

k-value	Success Percentage (%) (d=1)
5	96.775
6	97.450
7	98.100
8	97.950
9	98.150
10	98.350
11	98.550
12	98.600
13	98.550
14	98.750
15	98.700

Table 2 - Percentage of total success of the SVD-based algorithm for all digits (0-9) using k (number of basis functions) between 5 and 15.

k-value	Overall Success Percentage (%)
5	92.095
6	92.663
7	93.412
8	93.767
9	94.203
10	94.310
11	94.545
12	94.617
13	94.875
14	95.037
15	95.242

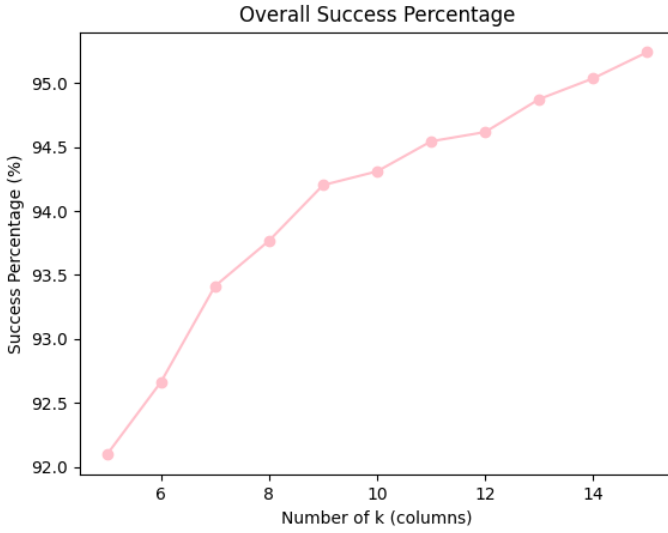


Figure 5 - Overall success percentage combined for all digits 0-9 for k 5-15.

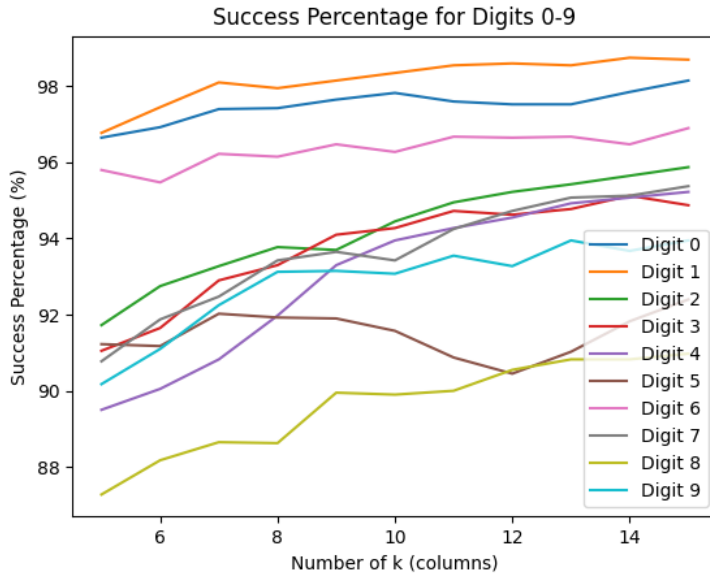


Figure 6 - Success percentage for all digits 0-9 for k 5-15.

Discussion

When we plotted the singular values for digits 3 and 8 using a linear scale for the x-axis and logarithmic scale for the y-axis (See Figure 1), we could observe that the initial singular values were significantly larger than the rest since the values then decayed rapidly until the values got close to zero at around the 650th singular value. The same thing can be observed in Figure 2, where the singular values also decay rapidly, in an exponential manner. This indicates that only a smaller number of the initial singular values are needed to conserve the information from the original data, which in this case is for computing images of digits. In other words, large singular values indicate where the variation of the data is the biggest and it is those parts of the data that

make up the main characteristics. This is confirmed by plotting the first three singular images for digits 3 and 8, where the digits could still be distinguished, despite not including all the original data for the plot (See Figure 3). The plots from u_1 , u_2 , and u_3 slightly differed, but there were similarities between the plots for the two digits. In the u_1 plot for the first singular image the shape of the external structure (outlines) of the digits were emphasized. Meanwhile smaller details of the digit structure, e.g. the curves and thickness, were highlighted in the other singular images. This may be due to noise and variation in how the digits can be written.

When plotting the first column of U_k for all digits, the basic and distinct shape of the digits could be seen (see Figure 4). This was expected since the first column of U_k corresponds to the first singular value, which we have confirmed is the most important one. So even though the dimension of the image has decreased significantly the image is still well-preserved. Thus, SVD can be used to reduce the dimensionality and decompose the data into smaller components while still maintaining the most essential parts of the images.

The algorithm was tested by using 2000 training images and then identifying images of $d = 1$ on 40 000 test images. Different numbers of basis functions k were used and the success rate for each correctly classified image was documented. The success rate increased as the number of basis functions k increased, but the increase slowed down gradually (See Table 1). This can be interpreted as that the first basis functions capture the most important parts of the data, and the latter only contribute with smaller features that are not as essential to the accuracy of the digits.

Then, we repeated this process for all ten digits and made a plot where each line represented the success rate for each digit (See Figure 6). The general pattern was that the success rate increased for all digits when the number of basis functions increased. This can also be seen in the figure displaying the overall success rate for all digits (See Figure 5). This could be explained by more singular vectors being used to construct the digit images, which increased accuracy. However, the success rate differed a bit between the digits, where digits 0 and 1 had the highest success rate overall, probably due to their structure being the most simplistic and therefore not requiring as many basis functions to be pictured accurately. Digits with more complex structures had a lower success rate for the same number of basis functions. A complex structure could include more curvatures, thickness, and other elements that cause more variation in how the digit can be written, as opposed to the straight line of digit 1 that has limited writing options which makes the structure less complex to reconstruct properly. So, depending on the complexity of the image, the number of basis functions needed to reconstruct the image with high accuracy varies.

Conclusion

Singular Value Decomposition (SVD) is an effective method for reducing the dimension of high-dimensional image datasets while still preserving the most significant properties of the images. It is important to determine which singular values contain the most significant data, which can be done by finding the largest singular values. In this project the dataset consisted of handwritten digits, and the digits all had a similar pattern of singular values, where the first singular values were significantly larger than the rest. The number of singular vectors needed for the reconstruction depends on the degree of accuracy needed, but generally a small number of

singular vectors are sufficient. In this project a low rank approximation of between five and fifteen was used which was enough to obtain a minimum of 85 % success rate (See Figure 6).